

BACKEND STATUS

Healthy

FastAPI / SQLAlchemy / SQLite & Postgres

FRONTEND STATUS

Active

Next.js 14 App Router · 8/8 routes
200 OK

WORKFLOW ASSETS

50 / 50

100% ticket journey assets present

FIGMA REDESIGN

~90%

Incoming client redesign in ~1 week

1. Executive Summary

50points is a tournament-based horse racing prediction platform powered by official, real-time results from United States racetracks. The core mechanic establishes that **1 ticket = 1 tournament = the last 7 races of the racetrack**. Unlike traditional race-by-race gambling, scoring is deterministic and directly tied to official racetrack payout dividends (*Win payoff*, base \$2.00) multiplied by chosen strategy weights (Full Point, Dual Point, or Smart Point).

Following stakeholder directives, this technical review was conducted in a non-destructive audit mode: the baseline was verified in main, dependencies and servers were built and executed in an isolated local environment, and zero architectural migrations or database modifications were made. The goal is to audit current runtime health, pinpoint reusable logic, and prepare for the upcoming client Figma overhaul.

2. System Architecture & Tech Stack

Layer	Technology Stack	Implementation & Audit Notes
Backend API	Python 3.12, FastAPI, Uvicorn	Modular routers (auth, tournaments, tickets, leaderboard, profile, races, chat, groups). Includes CORS and lifespan tasks.
Database & ORM	SQLAlchemy 2.0, PostgreSQL / SQLite	Clean models in <code>models.py</code> . Dev fallback to local SQLite (<code>dev.db</code> with WAL); production connects via Supabase IPv4 Session Pooler.
Frontend	Next.js 14.2 (App Router), React 18, Tailwind	App directory routes (<code>src/app/</code>) with modular client code in <code>src/frontend/</code> (components, contexts, and API wrappers).
Live Racing Data	The Racing API (North America) + HRN Scraper	Primary API uses regional endpoints (<code>/v1/north-america/meets</code>). Secondary HRN scraper acts as fallback. Results are strictly real.
Deployment	Railway (Backend) & Vercel (Frontend)	Containerized Railway deployment with background sync; auto Vercel deployment triggered from production branches.

3. Core Game Mechanics & Scoring Strategies

FULL POINT (50)

Selection: 1 Horse

Formula: $\text{Round}(50 \times \text{Dividendo})$

Maximum risk, maximum reward. Points awarded only if chosen horse wins.

DUAL POINT (25 + 25)

Selection: 2 Horses

Formula: $\text{Round}(25 \times \text{Dividendo})$

Balanced strategy across two runners. Win payoff applied to the winning slot.

SMART POINT (30 / 15 / 5)

Selection: 3 Horses

Formula: $\text{Round}(\text{Base_Slot} \times \text{Dividendo})$

Strategic distribution. Scratched picks transfer to post-time favorite.

4. Runtime Verification & Audit Findings

Audit Confirmation: 100% Non-Destructive Execution

Tests were run using local virtual environments and local ports (Backend: 8000, Frontend: 3000). The remote Git repository working tree remains clean and untouched.

BACKEND SERVICE VERIFICATION

- **Server Lifespan & Seeding:** Executed `python run.py --port 8000`. Initialized SQLite, applied missing columns, and seeded sample US racetracks.
- **Health & Endpoints:** GET `/health` returned 200 OK. GET `/api/tournaments` returned active tournaments and races with correct timezone conversions.
- **Authentication & Guest:** Tested POST `/api/auth/guest`. Generated unique guest credentials with 12h expiration JWT and guest token (50P-814495).
- **Scoring Simulation:** Verified `simulate_results.py`. Confirmed dividend calculations, leaderboard deltas, and streak updates.

FRONTEND WEB APP VERIFICATION

- **Build & Server:** Executed Next.js 14 dev server via `scripts/next-cli.mjs`. Resolved Windows path casing normalization automatically.
- **Route Testing (200 OK):** Validated `/`, `/tournaments`, `/tournament/[id]`, `/leaderboard`, `/how-to-play`, `/login`, `/register`, `/profile`.
- **Routing Structure:** Confirmed tournament detail is mounted on `/tournament/[id]` (singular), while the index is `/tournaments`.
- **Linting:** ESLint error found in `ProfileTicketHistoryPanel.jsx` (unescaped quotes), currently bypassed via `ignoreDuringBuilds`.

5. Static Assets Verification & Problem Areas

Component / Area	Status	Metrics	Findings & Recommendations
Ticket Workflow Assets	Complete	50 / 50 (100%)	All tabs, badges, stubs, and accordion connectors present in <code>public/Img/</code> .
How-to-Play Assets	Complete	10 / 10 (100%)	All step cards, icons, and infographics are present and functional.
Point Modality Assets	Partial	3 / 17 (18%)	14 cyberpunk hero graphics missing (defined in <code>POINT_MODALITY_ASSET_PROMPTS.md</code>).
Leaderboard Assets	Missing	0 / 9 (0%)	Gold/silver/bronze podiums, streak flames missing (in <code>LEADERBOARD_ASSET_PROMPTS.md</code>).
Git Branch Discrepancy	Action Req.	1 Remote Branch	Remote origin only has main. Neither development nor recovered-baseline tag exists on GitHub.
Payment Gateway (§14)	Blocked	Modalities 1 & 3	Paid ticket entry (~1 € and ~50 €) is hard-blocked until payment gateway (Stripe/Redsys) is integrated.
Figma Redesign (~90%)	In Progress	Incoming (1 wk)	Client is rebuilding 90% of UI. Avoid rewriting current frontend; focus on headless hooks & API decoupling.

6. Recommended Implementation Roadmap & Stakeholder Questions

IMPLEMENTATION ORDER

1. **Phase 1:** Align Git branches (development / recovered-baseline tag) and resolve ESLint entity escaping.
2. **Phase 2:** Decouple state hooks (useAuth, ticket builder) into headless hooks ready for new Figma UI.
3. **Phase 3:** Build modern Figma components section-by-section as client delivers them and wire to backend API.
4. **Phase 4:** Implement "Mis Tickets", dynamic "EN JUEGO" badges, and integrate Payment Gateway for paid modalities.
5. **Phase 5:** End-to-end tournament testing (onboarding → picks → scoring → leaderboard deltas).

QUESTIONS FOR STAKEHOLDERS (ADMIN / CLIENT)

1. **Branching:** Should developer push development and recovered-baseline to origin, or will tech lead push them?
2. **Figma Delivery:** Will designs arrive section-by-section (e.g. Onboarding first, then Tournament, then Leaderboard)?
3. **The Racing API:** Will staging credentials be provided, or should local development continue with SQLite / `simulate_results.py`?
4. **Payment Gateway:** Which payment provider is preferred for paid modalities (Stripe, Redsys/BBVA, PayPal, Crypto)?